Deep Reinforcement Learning

Offline Reinforcement Learning

Prof. Dr. Sebastian Peitz

Scientific Machine Learning group @ Paderborn University

Winter term 2026/27

PADERBORN UNIVERSITY

SciML

Content

Content

- Learning policies from datasets
- Offline reinforcement learning
  - Distribution shifts
  - Overestimation bias
  - Relation to imitation learning
  - Algorithmic evolution of methods
- Policy constraint methods
  - Forward and backward KL divergence
  - RL + BC (behavior cloning)
  - advantage-weighted regression
  - Batch-constrained Q-learning
- Conservative Q-learning
- Implicit Q-learning

Where are we?

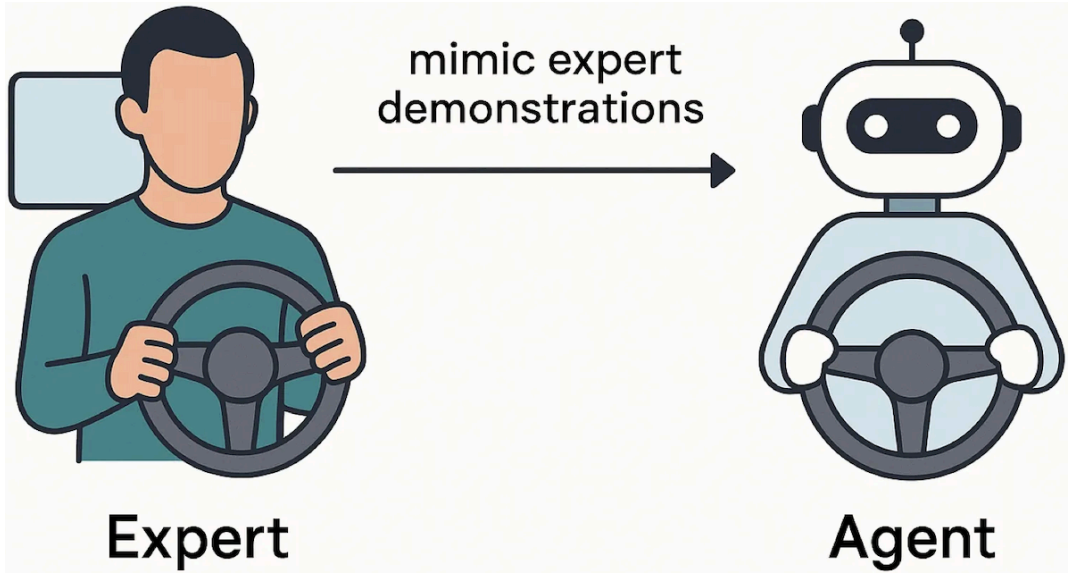
Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6-13	DQN, policy gradients, actor-critic, PPO/SAC, exploration	Deep RL from basics to modern algorithms
	Model-Based Control	
14-15	Optimal & feedback control, MPC, planning, model-based RL	Planning & control when we know the model
	Advanced Topics	
16	Offline reinforcement learning	RL with a static, given dataset

Lecture contents

Learning policies from datasets

Imitation learning

- Two of the biggest challenges in RL:
  1. Long waiting times until we find a policy that does not fail completely.
  2. Designing good reward functions that actually make the agent do what we want it to do.
- What can we do to circumvent this?
  ⇒ "Let's just show the agent how it's done?": learn from an expert!
- The concept behind this is **imitation learning**.
  - Given examples from a human expert, try to derive a policy that mimicks this behavior as closely as possible …
  - … while generalizing to previously unseen scenarios.



Source: *Medium.com*.



Source: *Innovation campus future mobility*.

Some developments in imitation learning (1)

Behavior cloning (BC)

- Treats imitation as **supervised learning**. Inputs: expert states; Outputs/Labels: expert actions as labels
- Ignores the underlying MDP.

+ Simple to implement.

+ No environment interaction during training.

+ Very fast.

− Compounding errors / covariate shift: A small error at step one puts the agent in a state it has never seen before, causing increasingly poor decisions.

− Assumes i.i.d. data distribution, which is false in sequential environments.

Inverse reinforcement learning (IRL)

- Instead of copying the actions, IRL tries to **figure out the intent** by assuming that the expert is optimizing an unobserved reward function.
- Extracts hidden reward function from the demonstrations and then runs standard RL on top.

+ Robust to compounding errors.

+ Reward function can transfer well even if the environment changes slightly.

− Expensive inner-loop problem: RL training in each iteration.

− Mathematically underdetermined (many different reward functions can explain the same expert behavior).

Some developments in imitation learning (2)

Interactive / Human-in-the-Loop IL

- To fix BC's compounding errors without the cost of IRL, methods like *Dataset Aggregation* (DAgger, (Ross, Gordon, and Bagnell 2011)) were introduced.
  - Runs current policy in the environment.
  - When it makes a mistake and goes off-track, an expert intercepts and provides the correct action labels for those erroneous states.
  - This data is added back into the training set.

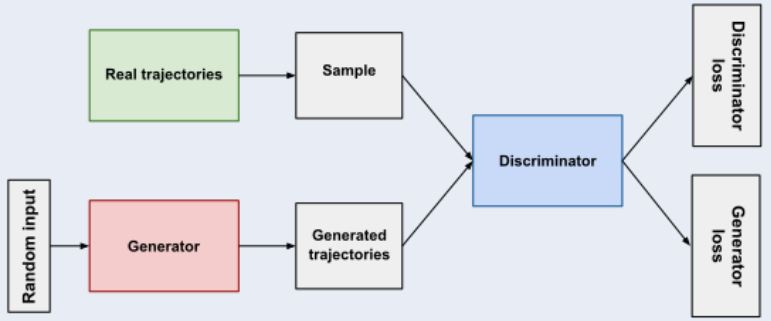
+ Directly fixes covariate shift.

+ Forces the agent to learn how to recover from its own mistakes.

– Requires an expert to be constantly available and online during training to label data.

Generative adversarial imitation learning (GAIL) (Ho and Ermon 2016)

- Two-player game:
  - Generator: the agent trying to mimic the expert policy.
  - Discriminator: classifier trying to distinguish between agent and expert trajectories.



Inspired by Generative Adversarial Networks (GANs).

- The discriminator penalizes the agent when it does not create expert-like trajectories.

+ Mimicks entire trajectories, not just single transitions as BC.

+ No expensive inner loop as in IRL.

+ Does not require an interactive expert like DAgger.

– Inherits GAN instabilities (mode collapse, highly sensitive hyperparameters).

— Exhausting for humans and frequently impossible for complex setups.

— Very data hungry.

What if we want to do better than the data?

Algorithm (class)	Key concept
Behavior cloning (BC)	Copy data (i.e., “expert” as closely as possible).
Inverse RL (IRL)	Identify reward from data ⇒ then standard RL.
Dataset aggregation (DAgger)	Expert corrects faulty behavior (meaning there is an online component).
Generative advarsarial imitation learning (GAIL)	The generative version of BC mimicking trajectories.

The key ideas in imitation learning

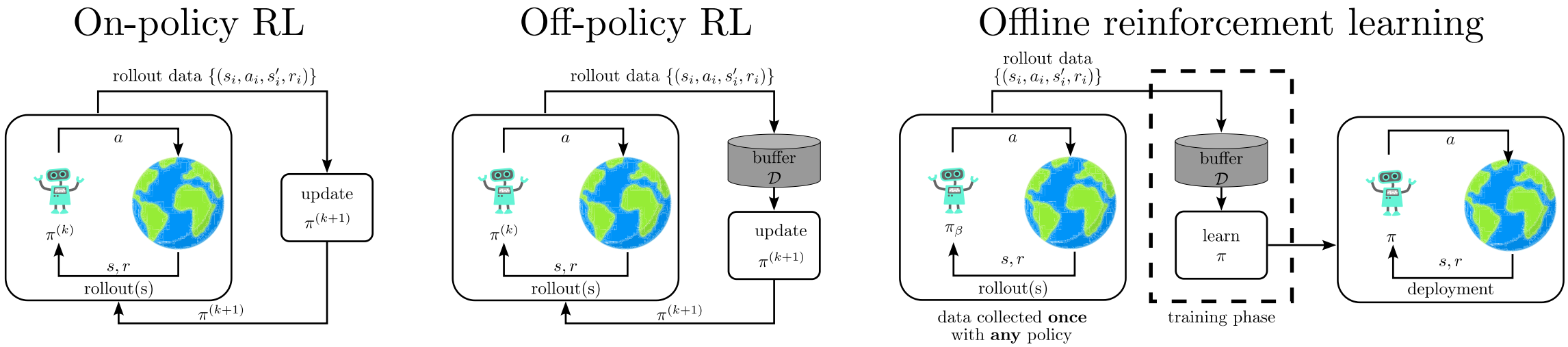
BUT: What if we cannot get new data, but we still want to be better than the performance of the dataset?
Maybe the data came from

- Experts,
- semi-experts,
- one or more past RL policies,
- a mixture of the above?

⇒ Offline reinforcement learning!

Offline Reinforcement Learning

What is offline RL?



Inspired by (Levine et al. 2020).

Offline RL is closely related to supervised learning / data-driven AI!

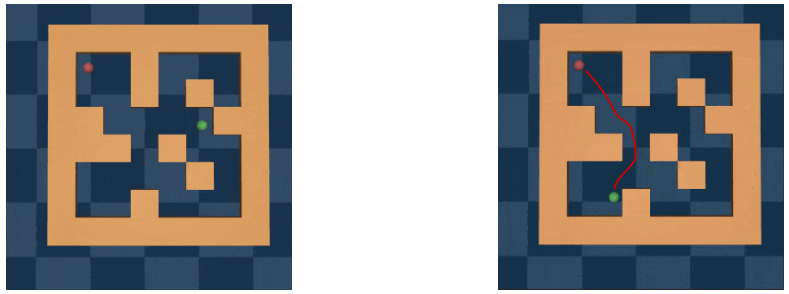
- Data-driven AI is about learning from large datasets.
  + Learns about the real world from data.
  — Doesn't try to do better than the data.
- Reinforcement learning is about optimization.
  + Optimizes a goal with emergent behavior.
  — Doesn't make use of real-world data.

⇒ Data without optimization doesn't allow us to solve new problems in new ways.
⇒ Optimization without data is hard to apply to the real world outside of simulators.

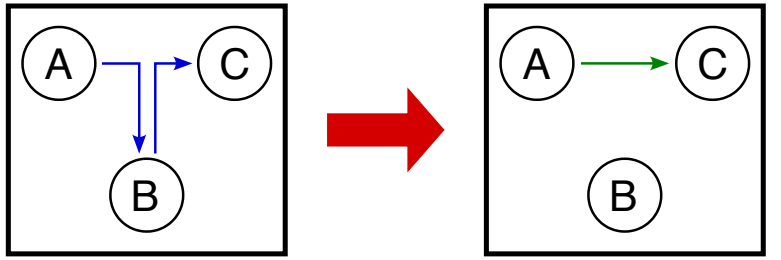
Offline reinforcement learning: Use *prior data* for RL training, *without any interactions* with the environment!
Goal: Given a dataset $\mathcal{D}$, learn the best possible policy π_ϕ that is *supported by the dataset*.

Can offline RL work?

1. **Find the "good stuff"** in a dataset full of good and bad behaviors.
   Example, consider a dataset of various drivers:
   ○ with imitation learning, we would find the average performance 👎
   ○ with offline RL, we hope to extract a policy mimicking the best driver's performance *in each situation* 👍
2. **Generalization**: good behavior in one place may suggest good behavior in another place.
   Driving example: we improve upon the best driver by generalizing to unseen situations.
3. **"Stitching"**: parts of good behaviors can be recombined.



Generalization (Source: D4RL).



Stitching (Source: Sergey Levine's CS285 lecture).

Takeaway: It's not just imitation learning!
Intuition: Get order from chaos.

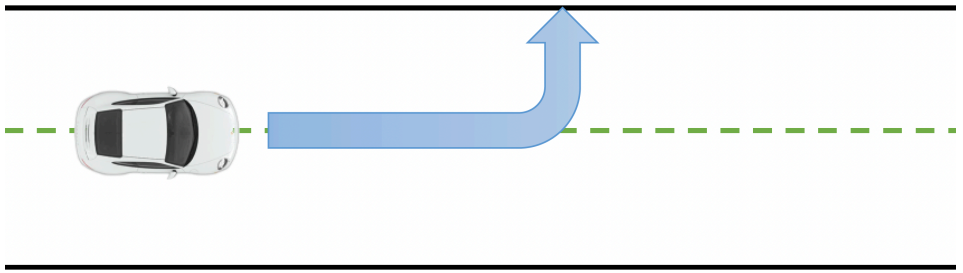
- On micro scales (e.g., a "straight path" from a bunch of wiggly trajectories).
- On macro scales (e.g., $A \to B \to C \Rightarrow A \to C$).

Why is offline RL so hard?

Fundamental problem: counterfactual queries



Training data.



What the policy wants to explore.

- Is this good or bad?
- How can we know if we have never seen it?

- **Online RL** algorithms don't have to handle this.
  - They can simply try this action and see what happens.
- **Offline RL** methods have to account for these unseen (*OOD*: "*out of distribution*") actions,
  - ideally in a safe way …
  - while still making use of generalization to come up with behaviors that are better than the best thing seen in the data!

Example: Q-learning / Bellman update

$$Q_\theta \leftarrow r + \gamma \mathbb{E}_{s' \sim p(s'|s,a)} \Big[\underbrace{\max_{a' \in \mathcal{A}} Q_{\bar{\theta}}(s', a')}_{\text{can go OOD}} \Big]$$

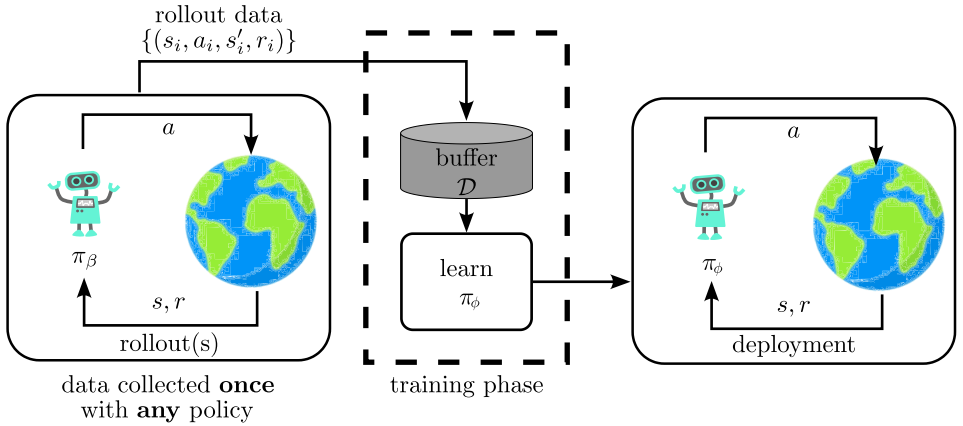
💡 ***out of distribution* vs. *out of sample*.**

- Out of sample ⇒ generalization: Evaluation outside the training dataset, but on data **from the same distribution**.
- Out of distribution ⇒ Evaluation on unseen **data from a different distribution**. This is much harder!

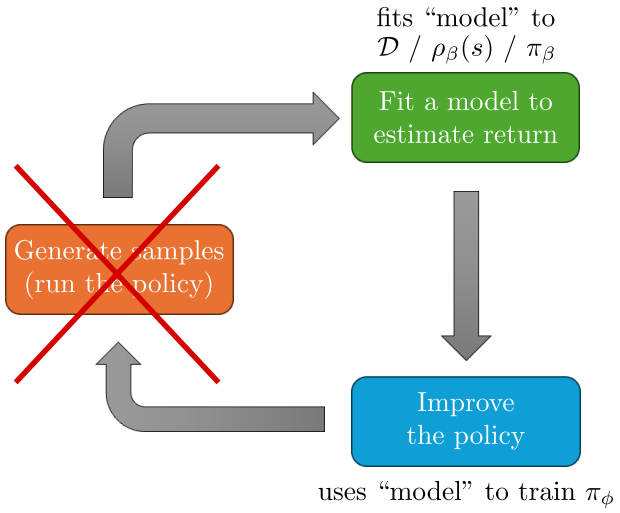
Inspired by Sergey Levine's CS285 lecture.

Distribution shifts (1)

Offline reinforcement learning



Inspired by (Levine et al. 2020).



Inspired by Sergey Levine's CS285 lecture.

Formally:

$$\mathcal{D} = \left\{(s_i, a_i, s'_i, r_i)\right\}_{i=1}^{N}$$
$$a \sim \pi_\beta(a|s) \quad \text{(generally unknown)}$$
$$s \sim \rho_\beta(s)$$
$$s' \sim p\left(s'|s, a\right)$$

RL objective: $\max_\phi \sum_{t=0}^{T} \mathbb{E}_{s\sim\rho_\phi, a\sim\pi_\phi(\cdot|s)} \left[\gamma^t r_t\right]$

- "Model" is valid under π_β...
- **not** under π_ϕ!
  - "Model" could be the Q-function Q_{π_ϕ},
  - or the dynamics $p_\phi(s'|s, a)$.

Distribution shifts (2)

Example: Empirical risk minimization (maximum likelihood)

$$\theta = \arg\min_{\hat{\theta}} \mathbb{E}_{x\sim p(x), y\sim p(y|x)} \left[\left(f_{\hat{\theta}}(x) - y \right)^2 \right]$$

Question: Given some x^*, is $f(x^*)$ correct?

- $\mathbb{E}_{x\sim p(x), y\sim p(y|x)} \left[\left(f_{\hat{\theta}}(x) - y \right)^2 \right]$ is low

  $\Rightarrow$ probably yes.

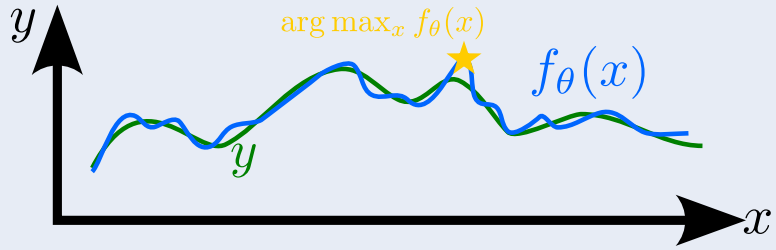
- $\mathbb{E}_{x\sim \bar{p}(x), y\sim p(y|x)} \left[\left(f_{\hat{\theta}}(x) - y \right)^2 \right]$ is generally not, if $\bar{p}(x) \neq p(x)$

  $\Rightarrow$ probably no.

- What if $x^* \sim p(x)$?
  - Not necessarily…
  - but neural networks tend to generalize well!

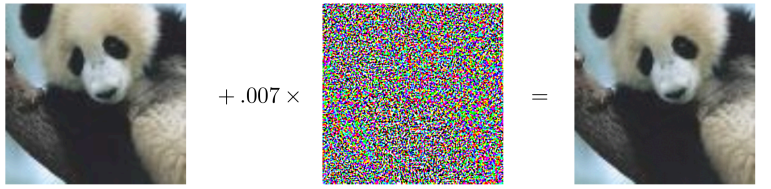
What if we pick the maximizing x^*?

$$x^* = \arg\max_x f_\theta(x)$$



Inspired by (Levine et al. 2020).

💡 This is exactly what we're exploiting in adversarial learning!



Adversarial example (Goodfellow, Shlens, and Szegedy 2015).

Overestimation bias in offline RL

Q-learning and Q-function actor-critic

$$\phi = \arg\min_{\hat{\theta}} \mathbb{E}_{s\sim\rho_\beta, a\sim\pi_\beta(\cdot|s)} \left[\| Q_\theta(s,a) - y \|^2 \right],$$

$$y_i = r_i + \gamma \mathbb{E}_{s_i'\sim\rho_\beta, a'\sim\pi_\phi(\cdot|s_i')} \left[Q_\theta(s_i', a') \right]$$

- Trying to find the best policy: **we want** $\pi_\phi(a|s) \neq \pi_\beta(a|s)$!
- But we only have s_i' for s_i, a_i from our offline dataset.
- **Even worse**: We're actually picking an *adversarial example*,

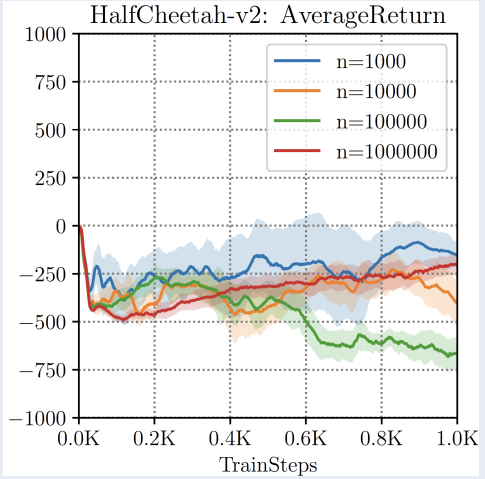
$$\phi = \arg\max_{\hat{\phi}} \mathbb{E}_{a\sim\pi_{\hat{\phi}}(\cdot|s)} \left[Q_\theta(s,a) \right].$$

Model-based RL

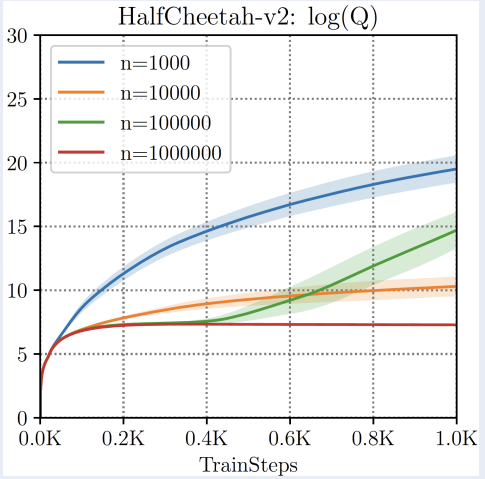
$$f = \arg\min_{\hat{f}} \mathbb{E}_{s\sim\rho_\beta, a\sim\pi_\beta(\cdot|s), s'\sim p(s'|s,a)} \left[\| f(s,a) - s' \|_2^2 \right]$$

- Probably large: $\mathbb{E}_{s\sim\rho_\beta, a\sim\pi_\phi(\cdot|s), s'\sim p(s'|s,a)} \left[\| f(s,a) - s' \|_2^2 \right]$.
- **Even worse**: Pick π_ϕ to *maximize the reward* under f!

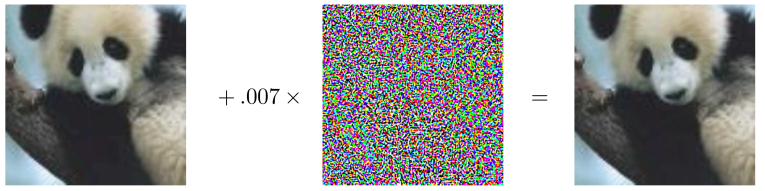
Example: SAC offline RL (Kumar et al. 2019)



How well the agent performs.



How well it thinks it does.



$\boldsymbol{x}$
"panda"
57.7% confidence

$\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$
"nematode"
8.2% confidence

$\boldsymbol{x} + \epsilon\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$
"gibbon"
99.3 % confidence

Adversarial example (Goodfellow, Shlens, and Szegedy 2015).

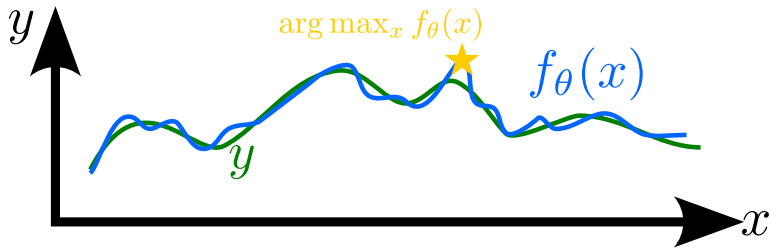
Algorithmic evolution of methods

The following strategies (or classes of approaches) are the most common ones today:

1. **Policy constraint methods**: Stay close to the data, e.g.,

$$D_{\text{KL}}\left(\pi_\phi(\cdot|s)\|\pi_\beta(\cdot|s)\right) \leq \epsilon.$$

2. **Value-regularization & pessimism**: Go anywhere, but assume what you haven't seen is dangerous ⇒ avoid overestimation.



Inspired by (Levine et al. 2020).

3. **Implicit Q-learning**: Implicitly define constraints to avoid leaving the support of the behavior policy π_β.

Common theme: "Maximize return while staying close to the dataset.

Policy constraint methods

Adding constraints

- Simplest approach to avoid overestimation: constrain the distance between behavior policy π_β and the learned policy π_ϕ.
- Remember the KL divergence?

$$D_{\text{KL}}\left(\pi_\beta(\cdot|s)\|\pi_\phi(\cdot|s)\right) = \mathbb{E}_{a\sim\pi_\beta(\cdot|s)}\left[\log\frac{\pi_\beta(a|s)}{\pi_\phi(a|s)}\right] = \mathbb{E}_{a\sim\pi_\beta(\cdot|s).}\left[\log\pi_\beta(a|s) - \log\pi_\phi(a|s)\right]$$

- Constraining the distance yields a constrained optimization problem:

$$\phi \leftarrow \arg\max_\phi \mathbb{E}_{s\sim\mathcal{D},a\sim\pi(\cdot|s)}\left[Q(s,a)\right] \qquad \text{s.t.} \qquad D_{\text{KL}}\left(\pi_\beta(\cdot|s)\|\pi_\phi(\cdot|s)\right) \leq \epsilon. \tag{1}$$

  - Where have we seen this before? ⇒ Natural policy gradient!

- Via Lagrange multipliers: additional term in the actor loss function:

$$\phi \leftarrow \arg\max_\phi \mathbb{E}_{s\sim\mathcal{D}}\left[\mathbb{E}_{a\sim\pi_\beta(\cdot|s)}\left[Q(s,a)\right] - \lambda D_{\text{KL}}\left(\pi_\beta(\cdot|s)\|\pi_\phi(\cdot|s)\right)\right]$$

$$\Rightarrow \quad \phi \leftarrow \arg\max_\phi \mathbb{E}_{s\sim\mathcal{D},a\sim\pi_\beta(\cdot|s)}\left[Q(s,a) + \lambda\log\pi_\phi(a|s) + \textbf{const}\right]. \tag{2}$$

  - To solve (1), we need to solve for both ϕ **and** the Lagrange multiplier λ.
  - Alternative: treat λ as a hyperparameter.

AC + BC methods

Methods of the type (2) are often termed "AC + BC": *actor-critic plus behavior cloning*, as the term $\log\pi_\phi(a|s)$ is typical in BC methods. Popular examples:

- TD3 + BC,
- DDPG + BC,
- SAC + BC.

Forward KL versus reverse KL

- Remember: the KL divergence is not a real metric. In particular, it violates the symmetry condition, i.e.,

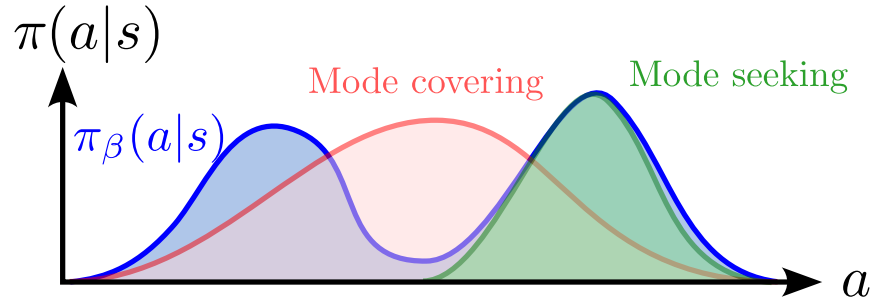
$$D_{\text{KL}}\left(\pi_\phi(\cdot|s)\|\pi_\beta(\cdot|s)\right) = \mathbb{E}_{a\sim\pi_\phi(\cdot|s)}\left[\log\frac{\pi_\phi(a|s)}{\pi_\beta(a|s)}\right] \neq \mathbb{E}_{a\sim\pi_\beta(\cdot|s)}\left[\log\frac{\pi_\beta(a|s)}{\pi_\phi(a|s)}\right] = D_{\text{KL}}\left(\pi_\beta(\cdot|s)\|\pi_\phi(\cdot|s)\right).$$

- **Forward KL** a.k.a. "mode covering":

$$D_{\text{KL}}\left(\pi_\beta(\cdot|s)\|\pi_\phi(\cdot|s)\right) = \mathbb{E}_{a\sim\pi_\beta(\cdot|s)}\left[\log\pi_\beta(a|s) - \log\pi_\phi(a|s)\right].$$

- **Reverse KL** a.k.a. "mode seeking":

$$\begin{aligned} D_{\text{KL}}\left(\pi_\phi(\cdot|s)\|\pi_\beta(\cdot|s)\right) &= \mathbb{E}_{a\sim\pi_\phi(\cdot|s)}\left[\log\pi_\phi(a|s) - \log\pi_\beta(a|s)\right] \\ &= -\mathbb{E}_{a\sim\pi_\phi(\cdot|s)}\left[\log\pi_\beta(a|s)\right] - \mathcal{H}(\pi_\phi(\cdot|s)). \end{aligned}$$



Mode seeking vs. mode covering (inspired by Sergey Levine's CS285 lecture).

- Mode covering: What happens if for some (s, a), $\pi_\phi(a|s) = 0$ while $\pi_\beta(a|s) > 0$?
  - KL divergence 💥 $\Rightarrow$ Need to cover everything from the dataset!
- Mode seeking: What happens if for some (s, a), $\pi_\beta(a|s) = 0$ while $\pi_\phi(a|s) > 0$?
  - KL divergence 💥 $\Rightarrow$ Need to ensure π_ϕ stays within a subset of π_β.

Which one is desirable?

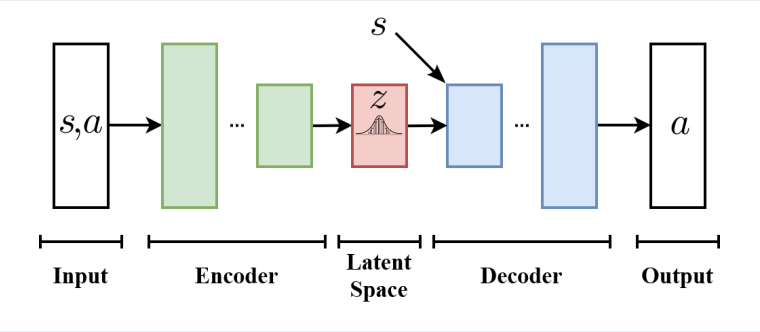
- Mode seeking is what we actually want.
- Extract the best behavior from a dataset.
- However, it is harder to realize, which is why we often use mode covering in practice, see *AC + BC*.

How to learn π_β if we don't know it

- Constraining the KL divergence requires knowledge of the behavior policy π_β.
- What can we do if we don't know it? $\Rightarrow$ extract it from the dataset!

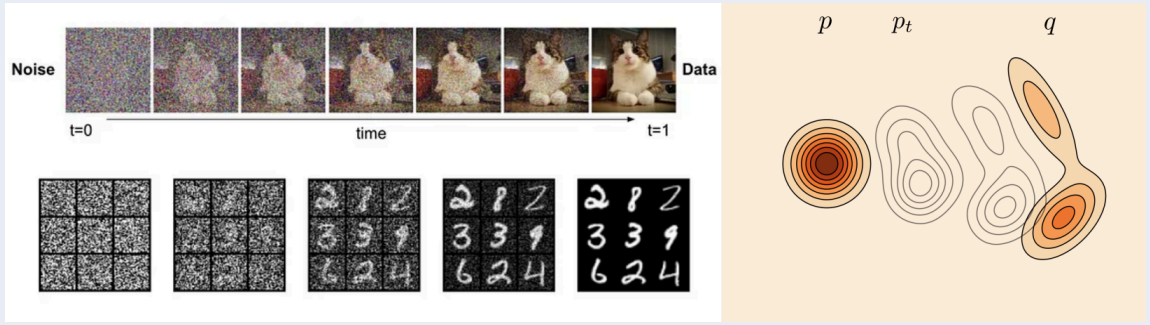
Behavior cloning.

- Given a dataset $\mathcal{D} = \{(s_i, a_i)\}_{i=1}^N$, we have a probabilistic supervised learning problem.
- Find the probability distribution $\pi_\psi(a|s)$ giving rise to $\mathcal{D}$.
- In other words maximize the likelihood of a_i given s_i over $\mathcal{D}$.
- Approaches:
  - Variational autoencoder (VAE) (Kingma and Welling 2013):



Adapted from Wikipedia.

  - Gaussian policy: $\pi_\psi(a|s) = \mathcal{N}(\mu_\psi(s), \sigma_\psi^2(s))$.
  - Generative modeling



Source: Medium.com.

Alternative to KL divergence

- The KL divergence is a natural choice for the policy constraint, but not necessarily the best one:

  + Mathematically tractable / closed-form solutions.
  + Direct penalization of OOD actions.
  + Smooth/soft regularization with tunable hyperparameter λ.

  — Asymmetry (forward KL vs. reverse KL).
  — Overly conservative in unseen regions.
  — Sensitive to behavior policy estimation.

- More generally, **we do not even want the two distributions to match**!
- We just want to find the best behavior supported by the dataset.
- Alternative: Minimizing Maximum Mean Discrepancy (MMD), see (Kumar et al. 2019):

$$MMD^2(\pi, \beta) = \mathbb{E}\left[k(a_\pi, a'_\pi)\right] - 2\mathbb{E}\left[k(a_\pi, a_\beta)\right] + \mathbb{E}\left[k(a_\beta, a'_\beta)\right].$$

  - k is a kernel function for implicitly evaluating inner products between feature vectors: $k(a_\pi, a_\beta) = \psi(a_\pi)^\top \psi(a_\beta)$.
  - $\mathbb{E}[k(a_\pi, a'_\pi)]$: How close to each other are the actions generated by our policy?
  - $-2\mathbb{E}[k(a_\pi, a_\beta)]$: How close are our actions to the dataset actions?
  - $\mathbb{E}[k(a_\beta, a'_\beta)]$: The internal variance of the dataset actions (a constant baseline).

  + Bypasses densitiy estimation / works on samples .
  + No infinite penalty on unsupported actions.
  + Allows for multimodel generalization.

  — Computationally expensive.
  — High sample variance / hard to tune the kernel bandwidth.
  — Curse of dimensionality.

The BRAC framework

Unification of various approaches in the BRAC framework

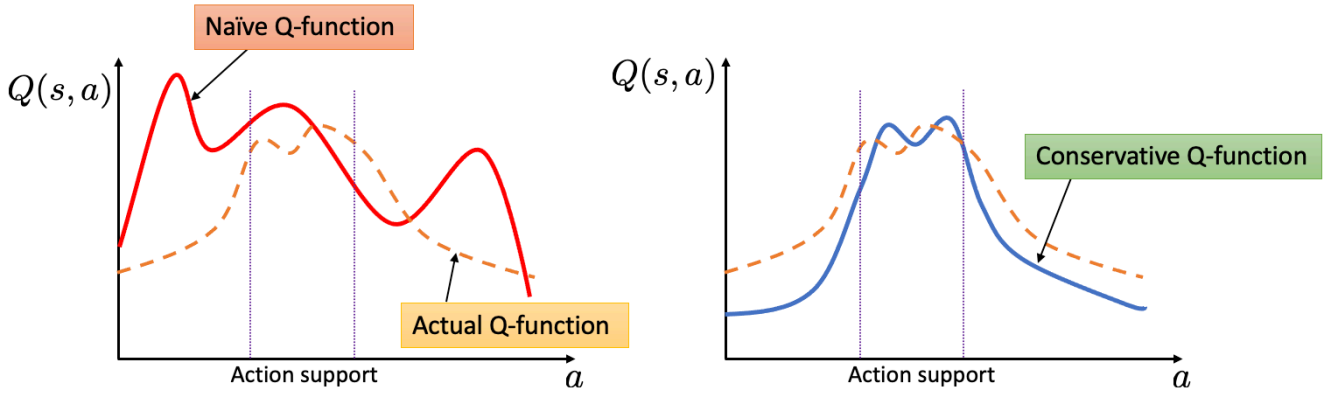
In (Wu, Tucker, and Nachum 2019), the authors present the *Behavior Regularized Actor-Critic (BRAC)*.

- This can be seen as a unification of many methods.
- The constraint can be enforced in two different ways
  - In the policy approximation (*policy regularization*)
  - In the value or Q function approximation (*value regularization*, coming up next)
- The constraint is defined as a general distance metric.
  - The KL divergence and MMD are then special cases.
- Additional design choice: How to approximate the expectation.

Value-regularization & pessimism

Conservative Q-learning

- We have seen that overestimation of Q-values outside the offline dataset $\mathcal{D}$ is the main source of failure for offline RL.
- *Policy constraint methods* (previously): Penalize deviations of the policy from the behavior policy π_β.
- Alternative: Penalize the Q-function outside $\mathcal{D} \Rightarrow$ **value regularization**!
- This is what **conservative Q-learning** (CQL, (Kumar et al. 2020)) does:
  $\Rightarrow$ introduce a lower bound (a pessimistic approximation) of the actual Q-function.
- We can then simply use the pessimistic Q-function in Q-learning or Actor-Critic methods.



Source: *Berkley AI Research (BAIR) blog.*

Standard actor-critic algorithm

1. Learn $\hat{Q}^\pi$ using offline dataset $\mathcal{D}$.
2. Optimize policy: $\pi \leftarrow \arg\max_\pi \mathbb{E}_\pi[\hat{Q}^\pi]$.

CQL algorithm

1. Learn $\hat{Q}^\pi_{\text{CQL}}$ using offline dataset $\mathcal{D}$.
2. Optimize policy: $\pi \leftarrow \arg\max_\pi \mathbb{E}_\pi[\hat{Q}^\pi_{\text{CQL}}]$.

Question: How can we achieve this?

The idea behind CQL

- How do we reduce the Q-function estimate outside our dataset?
  - Just reduce Q *everywhere*…
  - then increase it again in the *region supported by the dataset*!
- The Q-learning step with TD(0):

$$\theta \leftarrow \arg\min_\theta \frac{1}{2}\mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(\underbrace{r+\gamma\mathbb{E}_{a'\sim\pi_\phi(\cdot|s')}\left[Q_{\bar{\theta}}(s,a)\right]}_{y} - Q_\theta(s,a)\right)^2\right].$$

- Add a conservative penalty term:

$$\theta \leftarrow \arg\min_\theta \left(\frac{1}{2}\mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y - Q_\theta(s,a)\right)^2\right] + \alpha \underbrace{\max_\mu \mathbb{E}_{s\sim\mathcal{D},a\sim\mu(\cdot|s)}[Q_\theta(s,a)]}_{=\mathcal{R}}\right).$$

  - We need to find a policy μ that maximally reduces the estimate of Q_θ.
  - Resulting Q-function lower-bounds the true Q-function point-wise.
- This raises two issues:
  1. We don't want a reduction on the parts supported by $\mathcal{D}$.
  2. Computation of μ is an expensive optimization problem in itself!

The CQL(H) framework (Kumar et al. 2020)

- Reduce Q everywhere, boost it on the dataset:

$$\hat{\mathcal{R}} = \mathcal{R} - \mathbb{E}_{a\sim\pi_\beta(\cdot|s)}\left[Q_\theta(s,a)\right].$$

- Add KL-divergence regularizer $\Rightarrow$ closed-form solution (similar to SAC):

$$\hat{\mathcal{R}} = \mathbb{E}_{s\sim\mathcal{D}}\left[\log\left(\int_{\mathcal{A}} \exp(Q_\theta(s,a))\,\mathrm{d}a\right) - \mathbb{E}_{a\sim\pi_\beta(\cdot|s)}\left[Q_\theta(s,a)\right]\right].$$

- Lower bound on expected value:

$$\mathbb{E}_{a\sim\pi}\left[Q_\theta(s,a)\right] \leq V^\pi(s).$$

💡 We here use an on-policy estimator, as is used in continuous control algorithms such as SAC. A Q-learning version were we maximize over actions in the target y is equally possible (Kumar et al. 2020).

Algorithmic implementation of CQL(H)

- Penalty term on our Q-function approximation:

$$\theta \leftarrow \arg\min_{\theta} \frac{1}{2}\mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y - Q_\theta(s,a)\right)^2\right] + \alpha\mathbb{E}_{s\sim\mathcal{D}}\left[\log\left(\int_{\mathcal{A}} \exp(Q_\theta(s,a))\,\mathrm{d}a\right) - \mathbb{E}_{a\sim\pi_\beta(\cdot|s)}\left[Q_\theta(s,a)\right]\right].$$

- Challenge: Monte Carlo sampling of the integral over $\mathcal{A}$!
- Uniform sampling will likely miss the large-Q sections $\Rightarrow$ We need a suitable approximation from data.
- Proposal in (Kumar et al. 2020): A mixture of three sets
  1. **Uniform actions ($a \sim U(\mathcal{A})$)**: ensures exploratino of the entire action space; reduces overestimation peaks in distant regions of the action space.
  2. **Current policy actions ($a \sim \pi_\phi(\cdot|s)$)**: exploits the highest peaks in the current Q-function $\Rightarrow$ directly targets overestimations π_ϕ is trying to exploit.
  3. **Next-state policy actions ($a \sim \pi_\phi(\cdot|s')$)**: In the Bellman equation, the target value is calculated using the policy's action at the next state ($a' \sim \pi(s')$). If $Q(s',a')$ is overestimated, that error propagates back to $Q(s,a)$ via the TD update.
- We would have to perform importance sampling to go from $\int \exp(Q_\theta(s,a))\,\mathrm{d}a \approx \frac{1}{N}\sum_{i=1}^{N}\exp(Q_\theta(s,a_i))$ to this sum of three terms. However, we ignore this in practice:

$$I(s) = \log\left(\frac{1}{3}\left(\mathbb{E}_{a\sim U(\mathcal{A})}\left[\exp Q_\theta(s,a)\right] + \mathbb{E}_{a\sim\pi_\phi(\cdot|s)}\left[\exp Q_\theta(s,a)\right] + \mathbb{E}_{a\sim\pi_\phi(\cdot|s')}\left[\exp Q_\theta(s,a)\right]\right)\right),$$

$$\theta \leftarrow \arg\min_{\theta}\left(\frac{1}{2}\mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y - Q_\theta(s,a)\right)^2\right] + \alpha\mathbb{E}_{s\sim\mathcal{D}}\left[I(s) - \mathbb{E}_{a\sim\pi_\beta(\cdot|s)}\left[Q_\theta(s,a)\right]\right]\right).$$

Example: Complex gripping tasks

Source: *Berkley AI Research (BAIR) blog.*

Implicit Q-learning

What we have seen until now (and why we need something better)

Explicit Policy Constraints (e.g., BEAR (Kumar et al. 2019) or BRAC (Wu, Tucker, and Nachum 2019))

- Restrict the learned policy $\pi(a|s)$ to stay close to the behavior policy $\pi_\beta(a|s)$ that collected the dataset.
- Typically using a distance metric like KL-Divergence.

Drawbacks:

- Requires explicit modeling of the behavior policy π_β.
- If $\mathcal{D}$ comes from human demonstrators or multiple mixed policies, π_β is highly complex and multimodal.
- Accurately fitting it is very difficult, and any error in π_β propagates into the learned policy.

Conservative Q-Learning (CQL, e.g., (Kumar et al. 2020))

- Instead of constraining the policy, CQL modifies the objective to learn a conservative lower-bound Q-function.
- Penalty on the Q-values of out-of-distribution actions.

Drawbacks:

- CQL requires sampling OOD actions during training to penalize them, which makes it computationally heavy.
- Can be excessively conservative, reducing performance on tasks that require stitching together different good sub-trajectories.
- Very sensitive to hyperparameter tuning.

The motivating question behind implicit Q-learning

What if we never evaluate an out-of-distribution action during training at all?

Instead of constraining the policy explicitly or penalizing unseen actions, IQL constrains the optimization implicitly by evaluating the Q-function only on state-action pairs (s, a) that actually exist in the dataset.

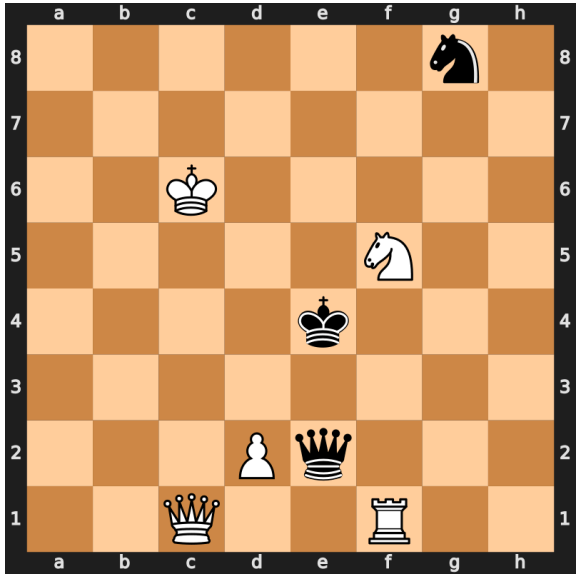
Benefits of IQL (Kostrikov, Nair, and Levine 2021):

- **No OOD action evaluation**: Avoids the maximization bias because the Q-function is never evaluated on unseen actions.
- **No behavior policy estimation**: No need to learn or approximate π_β, eliminating the main source of compounding errors.
- **Sub-trajectory stitching**: Enables the agent to take the best parts of different suboptimal trajectories and combine them into an optimal path.
- **Computational efficiency**: Because it only uses in-sample data, training is significantly faster and more stable than CQL.

Advantage-weighted regression

Let's consider the chess example

- Imagine we are trying to learn how to play chess purely by watching a large database of past games.
  - Some games were played by grandmasters, some by mediocre players or even beginners.
  - If we copy every move equally (standard *Behavior Cloning (BC)*), we will become a mediocre player because we are copying the mistakes of the beginners just as much as the great moves of the masters.
- Ideally, we want to copy the actions that led to *better-than-expected* outcomes and ignore (or suppress) the actions that led to poor outcomes.



Chess board [Source]

Advanted-weighted regression (AWR, (Peng et al. 2019)):

- Imitation learning on the dataset.
- State-action pairs are weighted by an exponential function of the "advantage" (i.e., how much better the action was compared to the average action from that state).
- Can be seen as *weighted behavior cloning* (*BC*).

Mathematical derivation of AWR

Remember trust region policy optmization (TRPO)?

- Optimize the policy, but don't let it move too far away from the current one!
- The main reason (besides avoiding large update steps): mathematical approximation.
  - We needed to take the expectation of our objective function w.r.t. the "wrong" distribution to make it work.
  - Instead of sampling from the new policy that we're optimizing, we took the expectation w.r.t. the old policy.
  - Only if these are not too far apart, were we allowed to do this (and capable of deriving a bound for the error).

The starting point of AWR

- Maximize the expected advantage $\eta(\pi)$ of our policy π over the baseline policy π_β:

$$\eta(\pi) = \mathbb{E}_{s\sim\rho^\pi, a\sim\pi(a|s)}\left[A^{\pi_\beta}(s,a)\right] = \mathbb{E}_{s\sim\rho^\pi, a\sim\pi(a|s)}\left[g - V^{\pi_\beta}(s)\right],$$

  - where $g\sum_{t=0}^{\infty}\gamma^t r_t$ is the return from a trajectory following π,
  - and the value $V^{\pi_\beta}(s)$ is the same return, only following the behavior policy π_β.
- We cannot sample from the state distribution $\rho^\pi \Rightarrow$ sample from the dataset $\mathcal{D}$ and optimize a *surrogate objective* $\hat{\eta}(\pi)$:

$$\hat{\eta}(\pi) = \mathbb{E}_{s\sim\rho^{\pi_\beta}, a\sim\pi(a|s)}\left[A^{\pi_\beta}(s,a)\right] = \mathbb{E}_{s\sim\mathcal{D}, a\sim\pi(a|s)}\left[g - V^{\pi_\beta}(s)\right].$$

- Similar to TRPO:

$$\pi^* = \arg\max_\pi \hat{\eta}(\pi) \qquad \text{s.t.} \qquad \bar{D}_{\mathsf{KL}}\left(\pi \| \pi_\beta\right) \leq \epsilon. \tag{3}$$

Solution to the AWR problem

- Method of Lagrange multipliers ⇒ closed-form analytical solution of (3):

$$\pi^*(a|s) \propto \pi_\beta(a|s) \exp\left(\frac{1}{\alpha} A^{\pi_\beta}(s,a)\right),$$

  with temperature hyperparameter α.
  - If $A > 0$: $\exp(\frac{1}{\alpha} A) > 1$ ⇒ The probability of picking this action increases relative to the dataset.
  - If $A < 0$: $\exp(\frac{1}{\alpha} A) < 1$ ⇒ The probability of picking this action decreases.
- To find a neural network policy $\pi_\phi(a|s)$, we approximate this optimal target policy by minimizing their KL divergence: $\min_\phi \bar{D}_{\text{KL}}(\pi^* \| \pi_\phi)$.
- Insert the analytical solution ⇒ $\pi_\beta(a|s)$ cancels out ⇒ **weighted maximum likelihood loss**:

$$L_\pi(\phi) = -\mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\exp\left(\frac{1}{\alpha} A^{\pi_\beta}\right) \log \pi_\phi(a|s)\right]. \tag{4}$$

- To compute the weight $\exp(\frac{1}{\alpha} A^{\pi_\beta}(s,a))$, we need the advantage.
  - Train a standard baseline value function $V_\theta(s)$ via MSE on the dataset $\mathcal{D}$:

The AWR algorithm

1. Fit value function approximation $V_\theta(s)$ to the data set $\mathcal{D}$:

$$\min_\theta L_V(\theta).$$

2. Optimize policy by approximating the analytically optimal policy:

$$\max_\phi L_\pi(\phi).$$

\+ Simple to implement.

– Single-step policy improvement.

– Because AWR relies on static dataset returns g rather than propagating future values via Bellman backups ($Q(s,a) = r + \gamma \max V(s')$), it cannot stitch trajectories.

$$L_V(\theta) = \mathbb{E}_{s\sim\mathcal{D}}\left[(g - V_\theta(s))^2\right]. \tag{5}$$

Implicit Q-learning (IQL)

Limitations of AWR

- It requires entire trajectories to compute Monte Carlo estimates of the value.
- It computes the *average/expected* value via the Monte Carlo approximation.
- As AWR relies on learning entire trajectory returns, it can only replicate the best full trajectories already present in the data.

Proposed changes in implicit Q-learning (IQL, (Kostrikov, Nair, and Levine 2021))

1. Use TD-learning instead of Monte Carlo sampling, i.e.,

$$L_{\text{TD}}(\theta) = \mathbb{E}_{(s,a,r,s',a')\sim\mathcal{D}}\left[r + \gamma Q_{\bar{\theta}}(s',a') - Q_\theta(s,a)\right] \qquad \text{(SARSA)}$$

$$\text{or} \qquad L_{\text{TD}}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[r + \gamma \max_{\hat{a}\in\mathcal{A}} Q_{\bar{\theta}}(s',\hat{a}) - Q_\theta(s,a)\right]. \qquad (Q\text{-learning})$$

2. Do not estimate the mean, but the best possible Q-function supported by the data:

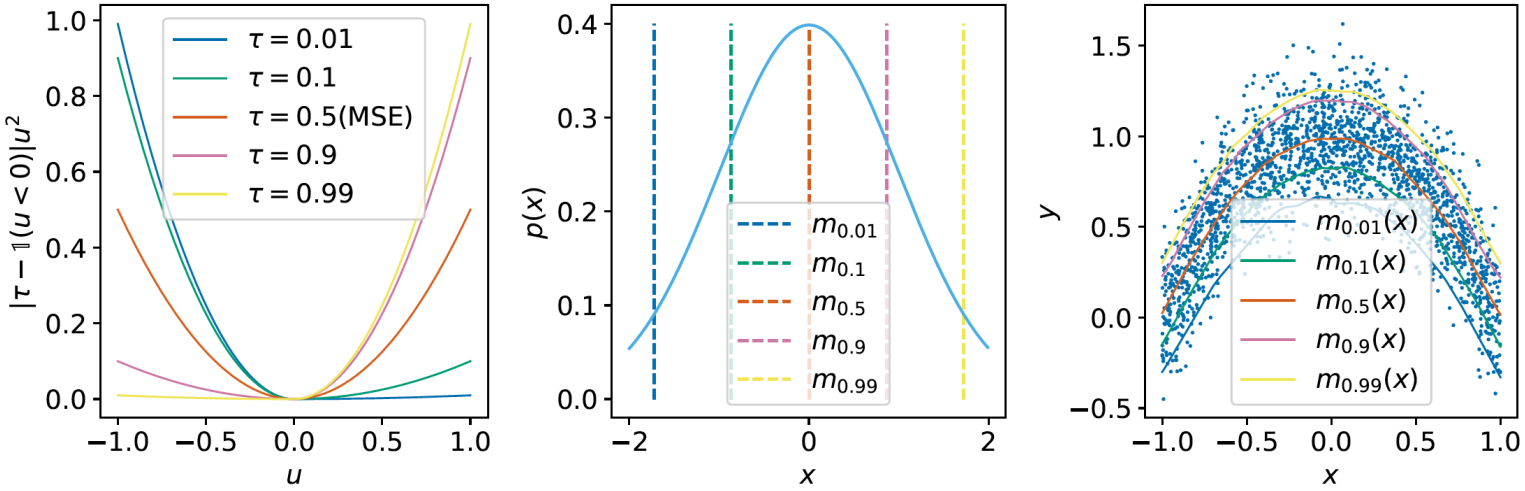
$$L_{\text{TD}}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[r + \gamma \max_{\substack{\hat{a}\in\mathcal{A} \\ \text{s.t. } \pi_\beta(\hat{a}|s')>0}} Q_{\bar{\theta}}(s',\hat{a}) - Q_\theta(s,a)\right].$$

Approach: Expectile regression plus AWR!

Expectile regression loss

- In statistical learning, it can be beneficial to perform *asymmetrical regression*.
- That is, we weight positive devaitions differently from negative deviations:

$$L_2^\tau(u) = |\tau - \mathbb{1}(u < 0)|\, x^2 = \begin{cases} (1-\tau)u^2 & \text{if } u > 0 \\ \tau u^2 & \text{if } u \leq 0 \end{cases}.$$



Left: *The asymmetric squared loss used for expectile regression. $\tau = 0.5$ corresponds to the standard mean squared error loss, while $\tau = 0.9$ gives more weight to positive differences.* ***Center***: *Expectiles of a normal distribution.* ***Right***: *an example of estimating state conditional expectiles of a two-dimensional random variable. Each x corresponds to a distribution over y. We can approximate a maximum of this random variable with expectile regression: $\tau = 0.5$ correspond to the conditional mean statistics of the distribution, while $\tau \approx 1$ approximates the maximum operator over in-support values of y. (Source: (Kostrikov, Nair, and Levine 2021, Figure 1)).*

The IQL algorithm

Implicit Q-learning (IQL, (Kostrikov, Nair, and Levine 2021))
Given: A dataset $\mathcal{D}$

- Estimate the value function by MSE optimization similar to (5), but using the **expectile loss** and a target Q network $Q_{\bar{\theta}}$:

$$L_V(\psi) = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[L_2^\tau(Q_{\bar{\theta}}(s,a) - V_\psi(s))\right]. \tag{6}$$

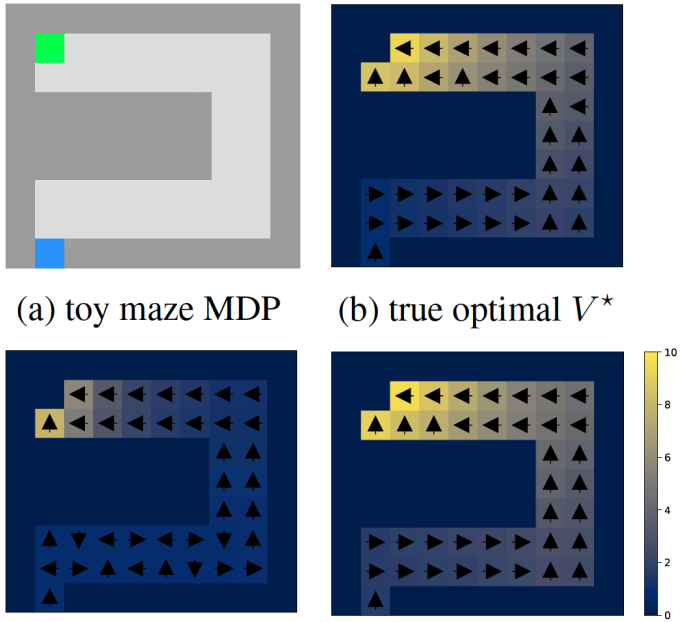
- Train a Q-network using the value network and TD learning:

$$L_Q(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[r + \gamma V_\psi(s') - Q_\theta(s,a)\right]. \tag{7}$$

- Train the policy network similar to AWR (Eq. (4)):

$$L_\pi(\phi) = -\mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\exp\left(\frac{1}{\alpha}Q_{\bar{\theta}}(s,a) - V_\psi(s)\right)\log\pi_\phi(a|s)\right]. \tag{8}$$

Note: If we can get access to new data, the above procedure can be repeated iteratively with incoming data, after updating the *replay buffer* $\mathcal{D}$.



(a) toy maze MDP (b) true optimal $V^\star$

(c) One-step Policy Eval. (d) IQL

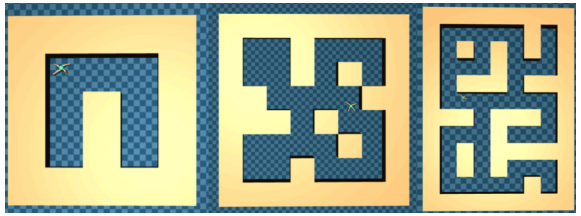
IQL on a toy umaze environment (a). When the data is heavily corrupted by suboptimal actions, one-step policy evaluation results in a value function that degrades to zero far from the rewarding states too quickly (c). IQL learns a near-optimal value function, combining the best properties of SARSA-style evaluation with the ability to perform multi-step dynamic programming, leading to value functions that are much closer to optimality (shown in (b)) and producing a much better policy (d). (Source: (Kostrikov, Nair, and Levine 2021, Figure 2)).

💡 In principle, we could also train (7) directly, without training a separate value network via (6). But the authors of (Kostrikov, Nair, and Levine 2021) argue that this can lead to the exploitation of "lucky samples", whereas the value averages out such conincidences.

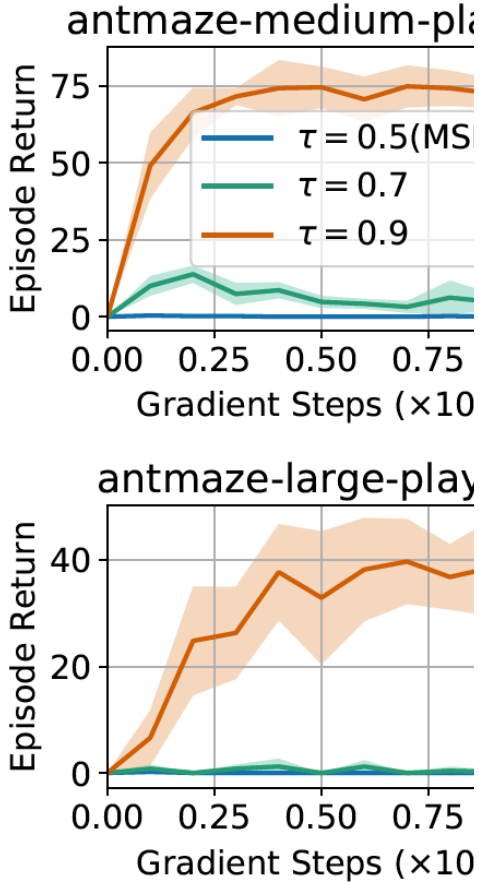
Some results

Dataset	BC	10%BC	BCQ	DT	ABM	AWAC	Onestep RL	TD3+BC	CQL	IQL (Ours)
halfcheetah-m-v2	42.6	42.5	**47.0**	42.6±0.1	53.6	43.5	**48.4±0.1**	**48.3±0.3**	44.0±5.4	**47.4±0.2**
hopper-m-v2	52.9	56.9	56.7	**67.6±1.0**	0.7	57.0	59.6±2.5	59.3±4.2	58.5±2.1	**66.2±5.7**
walker2d-m-v2	75.3	75.0	72.6	74.0±1.4	0.5	72.4	**81.8±2.2**	83.7±2.1	72.5±0.8	78.3± 8.7
halfcheetah-m-r-v2	36.6	40.6	40.4	36.6±0.8	**50.5**	40.5	38.1±1.3	**44.6±0.5**	**45.5±0.5**	**44.2±1.2**
hopper-m-r-v2	18.1	75.9	53.3	82.7±7.0	49.6	37.2	**97.5±0.7**	60.9±18.8	**95.0±6.4**	**94.7±8.6**
walker2d-m-r-v2	26.0	62.5	52.1	66.6±3.0	53.8	27.0	49.5±12.0	**81.8±5.5**	77.2±5.5	73.8±7.1
halfcheetah-m-e-v2	55.2	**92.9**	**89.1**	86.8±1.3	18.5	42.8	**93.4±1.6**	**90.7±4.3**	**91.6±2.8**	86.7±5.3
hopper-m-e-v2	52.5	**110.9**	81.8	**107.6±1.8**	0.7	55.8	103.3±1.9	98.0±9.4	**105.4±6.8**	91.5±14.3
walker2d-m-e-v2	**107.5**	**109.0**	**109.5**	**108.1±0.2**	3.5	74.5	**113.0±0.4**	**110.1±0.5**	**108.8±0.7**	**109.6±1.0**
locomotion-v2 total	466.7	**666.2**	602.5	**672.6±16.6**	231.4	450.7	**684.6±22.7**	**677.4±44.5**	**698.5±31.0**	**692.4±52.1**
antmaze-u-v0	54.6	62.8	**89.8**	59.2	59.9	56.7	64.3	78.6	74.0	**87.5 ± 2.6**
antmaze-u-d-v0	45.6	50.2	**83.0**	53.0	48.7	49.3	60.7	71.4	**84.0**	62.2 ± 13.8
antmaze-m-p-v0	0.0	5.4	15.0	0.0	0.0	0.0	0.3	10.6	61.2	**71.2 ± 7.3**
antmaze-m-d-v0	0.0	9.8	0.0	0.0	0.5	0.7	0.0	3.0	53.7	**70.0 ± 10.9**
antmaze-l-p-v0	0.0	0.0	0.0	0.0	0.	0.0	0.0	0.2	15.8	**39.6±5.8**
antmaze-l-d-v0	0.0	6.0	0.0	0.0	0.0	1.0	0.0	0.0	14.9	**47.5±9.5**
antmaze-v0 total	100.2	134.2	187.8	112.2	109.1	107.7	125.3	163.8	303.6	**378.0±49.9**
total	566.9	800.4	790.3	784.8	340.5	558.4	809.9	841.2	1002.1	**1070.4±102.0**
kitchen-v0 total	**154.5**	-	-	-	-	-	-	-	144.6	**159.8±22.6**
adroit-v0 total	104.5	-	-	-	-	-	-	-	93.6	**118.1±30.7**
total+kitchen+adroit	825.9	-	-	-	-	-	-	-	1240.3	**1348.3±155.3**
runtime	10m	10m		960m	20m	20m*	20m	80m	20m	

Averaged normalized scores on MuJoCo locomotion and Ant Maze tasks (Source: (Kostrikov, Nair, and Levine 2021, Table 1)).



Ant maze in various sizes (small, medium, large).



The influence of the expectile loss (Source: (Kostrikov Levine 2021, Figure 3)).

Summary / what we have learned

- Offline RL: Learn policies from a fixed, unchangeable dataset $\mathcal{D}$
- Main challenges:
  - Distribution shift between learned policy π and dataset policy π_β
  - Overestimation of Q outisde the dataset $\mathcal{D}$
- Approaches to address this:
  - Constrain the policy $\Rightarrow$ Explicit policy constraint methods (e.g., AC + BC)
  - Pessimistically modify the Q-function outside $\mathcal{D}$ $\Rightarrow$ Conservative Q-learning
  - Avoid out-of-distribution actions altogether $\Rightarrow$ Implicit Q-learning

References

Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy. 2015. “Explaining and Harnessing Adversarial Examples.” In *International Conference on Learning Representations (ICLR)*.

Ho, Jonathan, and Stefano Ermon. 2016. “Generative Adversarial Imitation Learning.” In *Advances in Neural Information Processing Systems*, edited by D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett. Vol. 29. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2016/file/cc7e2b878868cbae992d1fb743995d8f-Paper.pdf.

Kingma, Diederik P, and Max Welling. 2013. “Auto-Encoding Variational Bayes.” *arXiv:1312.6114*.

Kostrikov, Ilya, Ashvin Nair, and Sergey Levine. 2021. “Offline Reinforcement Learning with Implicit Q-Learning.” In *Deep RL Workshop NeurIPS 2021*.

Kumar, Aviral, Justin Fu, Matthew Soh, George Tucker, and Sergey Levine. 2019. “Stabilizing Off-Policy Q-Learning via Bootstrapping Error Reduction.” *Advances in Neural Information Processing Systems* 32.

Kumar, Aviral, Aurick Zhou, George Tucker, and Sergey Levine. 2020. “Conservative Q-Learning for Offline Reinforcement Learning.” In *Advances in Neural Information Processing Systems*, edited by H. Larochelle, M.